

Prompt-1

Now I am building a CShell so let's discuss about the architecture about the initial setup i) base setup which takes input ii) user can type input ii) validation so can you tell the both high level and low level design

Thought for 17s

Nice project — building your own shell in C is one of the best ways to learn systems programming. Let's break down the architecture for the initial phase you mentioned: setup → read input → validate.

High-Level Design

Think of your shell as a **REPL loop** (Read-Eval-Print-Loop) with distinct, decoupled stages:

```
graph LR
    subgraph Loop [REPL Loop]
        direction LR
        Init["Initialize (once)"] --> Print["Print Prompt (loop start)"]
        Print --> Read["Read Input"]
        Read --> Valid["Validate"]
        Valid --> Exec["Execute"]
        Exec --> Print
    end
    Valid -- "EOF/Ctrl-D" --> LoopBack["loop back"]
    Valid -- "invalid" --> RePrompt["print error, re-prompt"]
    LoopBack --> Print
    RePrompt --> Print
```

Key architectural principle at this stage: **each stage should be a pure-ish function with a single responsibility**, so later you can swap tokenizer implementations, add pipes, redirection, etc., without touching the input/validation code.

Key architectural principle at this stage: **each stage should be a pure-ish function with a single responsibility**, so later you can swap tokenizer implementations, add pipes, redirection, etc., without touching the input/validation code.

Modules for phase 1:

1. **Shell Core** — `main()`, owns the loop, calls other modules in sequence
2. **Input Handler** — reads a raw line from stdin, handles EOF/interrupts
3. **Validator** — checks the raw (or lightly tokenized) input for structural problems before you even try to parse/execute it

Low-Level Design

i) Base Setup

```

c
// shell.h
#define MAX_INPUT_SIZE 1024 // fallback only if you don't use getline

typedef enum {
    VALID,
    ERR_EMPTY_INPUT,
    ERR_TOO_LONG,
    ERR_UNMATCHED_QUOTE,
    ERR_INVALID_LEADING_OPERATOR,
    ERR_UNKNOWN
} ValidationStatus;

void init_shell(void);
void print_prompt(void);
char *read_input(void);
ValidationStatus validate_input(const char *input);

```

```

c
// main.c
int main(void) {
    init_shell();

    char *line;
    while (1) {
        print_prompt();
        line = read_input();

        if (line == NULL) { // EOF (Ctrl+D)
            printf("\n");
            break;
        }

        ValidationStatus status = validate_input(line);
        if (status != VALID) {
            report_error(status);
            free(line);
            continue;
        }

        // next phase: tokenize + execute(line)
        free(line);
    }
    return 0;
}

```

Prompt2

here when i mean design not the code like the things i should use

Got it — let's talk in terms of *concepts and components*, not code. Here's what to actually reach for and why, at each stage.

i) Base Setup

What it needs to establish before the loop starts:

- **Signal handling strategy** — decide how **SIGINT** (Ctrl+C) and **SIGQUIT** (Ctrl+) should behave. In a real shell, these shouldn't kill the shell process itself — they should either be ignored or restored to default only inside a forked child. This is a design decision you make *once*, at startup, not per-command.
- **Terminal/environment inspection** — things like whether you're running interactively (**isatty()**-style check) vs. reading from a script/pipe. This changes whether you show a prompt at all.
- **Shell state/context object** — even a "toy" shell benefits from a single struct that holds global state: current working directory, exit status of last command, shell PID, whether it's interactive. Avoid global scattered variables — group them into one context you pass around or hold in a well-defined place. This pays off massively once you add job control.
- **Config/rc file loading** — optional at this stage, but architecturally this is where it'd go (read **.myshrc**-equivalent before entering the loop).

ii) Reading Input

The core things to decide:

- **Dynamic vs fixed buffer** — the real design decision is: do you want a growable buffer (so arbitrarily long input never truncates or overflows) or a bounded one with explicit length-checking? A growable-buffer approach is the standard for real shells.
- **Line-based abstraction** — input reading should hand back "one logical line," not raw bytes. Keep the *reading* concern (bytes from stdin) separate from the *parsing* concern (breaking that line into meaning). This separation is the single most important design choice in this whole system — everything downstream (validation, tokenizing, expansion) assumes it receives a clean logical line.
- **Memory ownership model** — decide upfront: does the input-reading component allocate and hand ownership to the caller, or does it reuse an internal buffer? Pick one convention and apply it consistently everywhere in the shell, since you'll be

allocating a lot (tokens, argv arrays, expanded paths) and inconsistent ownership is the #1 source of leaks/double-frees in shell projects.

- **Interactive vs non-interactive distinction** — the same "read input" concept behaves differently if it's showing a prompt and reading a terminal line vs. silently consuming a script file. Design the input layer so it doesn't care — feed it a generic input source abstraction (could be stdin, could be a file) rather than hardcoding terminal behavior in.
- **Line continuation** — worth deciding early whether you'll support trailing-backslash or unclosed-quote line continuation (multi-line commands). If yes, "read input" isn't just one `getline` call — it's a small state machine that keeps reading until it has a *complete* logical command.

iii) Validation

Think of this as a pipeline of independent checks, not one big function:

- **Structural vs semantic validation** — separate "is this syntactically well-formed" (balanced quotes, no operator in illegal position, not empty) from "does this actually make sense to execute" (does the command exist, are arguments valid). Only the first belongs at this stage. The second belongs at execution time. Mixing these two is a common design mistake that makes your validator bloated and your error messages misleading.
- **Rule-based/composable checks** — instead of one function with cascading if-else, think of validation as a **list of independent predicate functions**, each responsible for one class of error (empty input, unmatched quotes, illegal leading operator, unterminated escape). This lets you add new rules (once you support `|`, `>`, `&&`) without touching existing ones — an "open for extension, closed for modification" design.
- **Error reporting as data, not strings** — return a status/error code (enum) from validation, and let a separate error-reporting component translate that into a user-facing message. This keeps validation logic decoupled from presentation, which matters once you want different behaviors for interactive vs scripted mode (e.g., scripts might want to abort entirely on validation failure, interactive mode just reprompts).
- **Where quote/escape tracking lives** — decide whether the validator does a lightweight character scan (just checking balance) or whether it reuses the *same* tokenizing logic the parser will use later. Reusing the same low-level scanner for both validation and tokenizing avoids duplicated, subtly-inconsistent parsing logic — one common approach is to have a shared "lexer" component that both the validator and tokenizer sit on top of.

The single biggest architectural idea to take away: treat this as a **pipeline of narrow, single-purpose components** (input source → lexer/scanner → validator → tokenizer → parser → executor), each with a clearly defined input/output contract and ownership rules, rather than one big loop doing everything inline. Everything you're planning to add later (pipes, redirection, background jobs, job control) just becomes new components slotted into this same pipeline — not rewrites of existing ones.

Prompt3 :

Can you explain and generate the MakeFile

```
CFLAGS = -std=c23 \
        -D_POSIX_C_SOURCE=200809L \
        -D_XOPEN_SOURCE=700 \
        -Wall -Wextra -Werror \
        -Wno-unused-parameter \
        -fno-asm

CPPFLAGS = -Iinclude

TARGET = shell.out

SRC = src/main.c \
      src/prompt.c \
      src/input.c

OBJ = $(SRC:.c=.o)

.PHONY: all clean

all: $(TARGET)

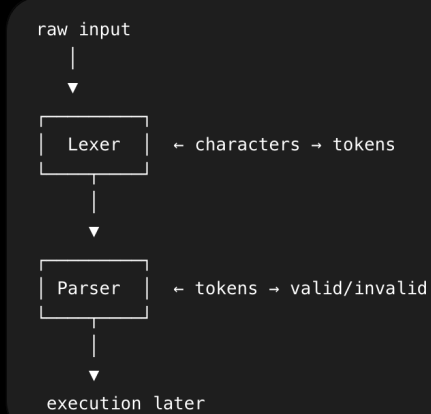
$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) $(CPPFLAGS) $(OBJ) -o $(TARGET)

%.o: %.c
    $(CC) $(CFLAGS) $(CPPFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ) $(TARGET)
```

Prompt4 :

Give me some ideas on Implementing A3:



Don't mix lexer and parser. That will make debugging much easier and also gives you a clean explanation for the TA.

Step 1: Design our A3 files

Your current structure can become:

```
c-shell/
├─ src/
│   ├── main.c
│   ├── prompt.c
│   ├── input.c
│   ├── lexer.c
│   └── parser.c
└─ include/
```

1. First understand what the lexer has to do

Suppose the user enters:

```
cat file.txt | grep hello > out.txt
```

The lexer should produce:

```
WORD("cat")
WORD("file.txt")
OP_PIPE
WORD("grep")
WORD("hello")
OP_GT
WORD("out.txt")
```

Notice:

cat

is not a **CAT** token.

It is:

WORD

Similarly:

file.txt

grep

hello

out.txt

are all **WORD**.

Operators

These are separate tokens:

| → OP_PIPE

& → OP_AMP

; → OP_SEMI

< → OP_LT

> → OP_GT

>> → OP_GTGT

And **maximal munch** means:

>>

must become:

OP_GTGT

not:

OP_GT

OP_GT

2. The most important lexer idea

A **WORD** isn't simply a sequence of ordinary characters.

It's:

$\text{WORD} \rightarrow \text{fragment}^+$

A fragment can be:

ordinary

or

`\any_character`

or

`"..."`

or

`'...'`

Therefore:

`hello"world"abc`

is **one WORD**:

`WORD("helloworldabc")`

Likewise:

`my\ file.txt`

is:

`WORD("my file.txt")`

And:

echo "hello world"

becomes:

WORD("echo")

WORD("hello world")

This distinction will be extremely important when we implement the lexer.

```
#ifndef LEXER_H
#define LEXER_H

#include <stdbool.h>
#include <stddef.h>

typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_AMP,
    TOKEN_SEMI,
    TOKEN_LT,
    TOKEN_GT,
    TOKEN_GTGT
} TokenType;

typedef struct Token {
    TokenType type;
    char *value;

    struct Token *next;
} Token;

Token *lex_line(const char *line);
void free_tokens(Token *tokens);

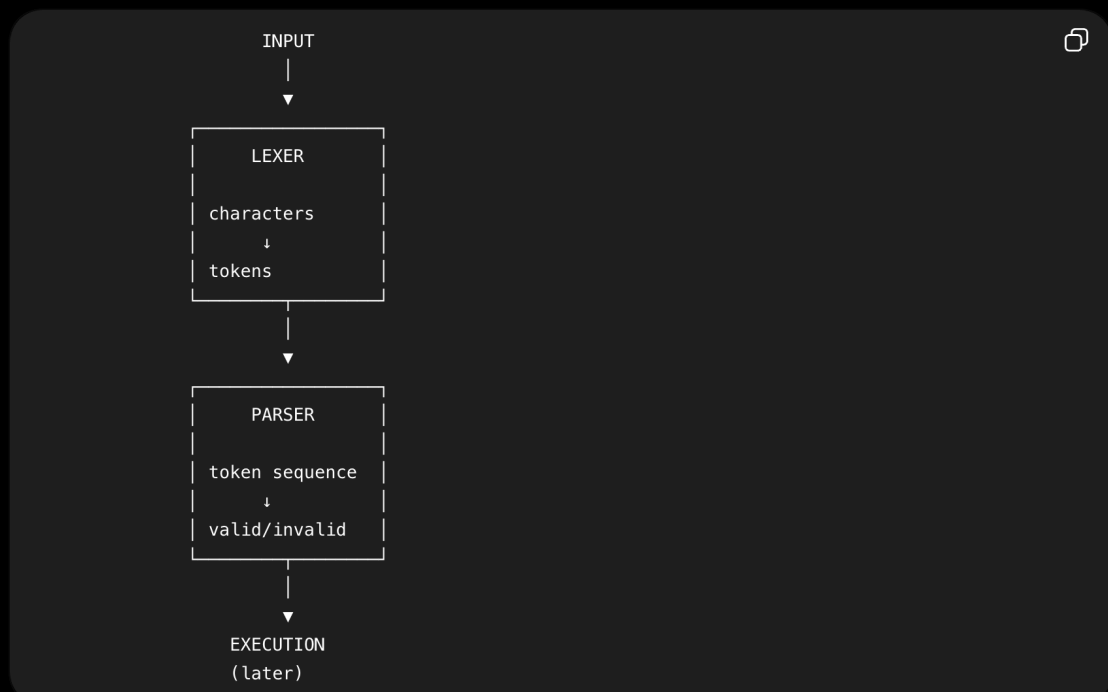
void print_tokens(const Token *tokens);

#endif
```



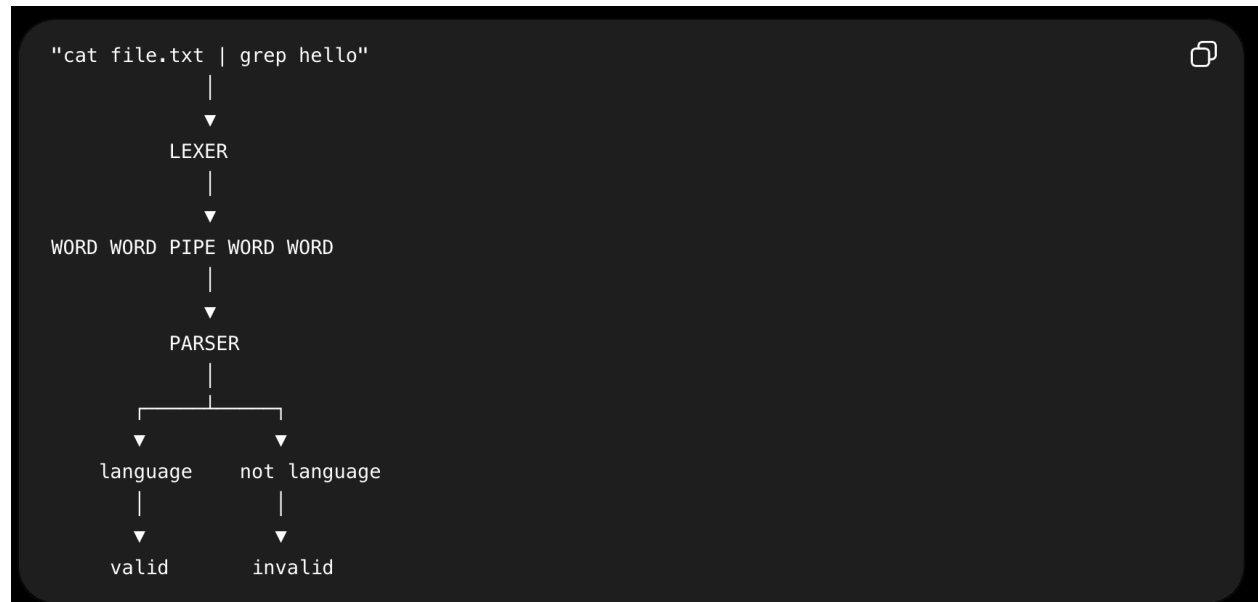
23. Our A3 architecture now

We're building this:



Prompt 5 :

hey now I have implemented lexer and want to implement parser . Once i will tell you my understanding then u constructiely build -> I understood displaying prompt which tells the user that it is ready for input and when input is taken ,lexer categorises the each token using the given conditions and also solves two synatx error not closed double q and / at end and as lexer only can solve basic syntax issues to check weather they are symatically correct or not we will use parseer ,here we will use the AT any input should be the string of the given Language i.e any input should be derivable from the production rules else it is not correct command .



Prompt 6 :

before moving to the parser can You take some example of some input and explain how it is categorized

```
echo
↓
WORD("echo")

space
↓
discard

"hello world"
↓
double quote fragment
↓
WORD("hello world")

space
↓
discard

>
↓
OP_GT

space
↓
discard

out.txt
↓
WORD("out.txt")
```

Prompt 7 :

kk understood so now we will check weather it is a valid sequnce or not can we do something like this because we are provided with the right linear grammar we will generate all the strings of the lenght which is equal to the length of our sequence

The better idea: derive only the string we actually have

This is where **parsing** comes in.

Instead of:

"Generate every possible string and see whether mine exists."

we do:

"Given my current token, which grammar production could have generated it?"

That's much smarter.

So think of it like a maze

Your grammar gives us a maze:



Prompt 8 :

I have written parser.h and parser.c check once `#ifndef PARSER_H`

```
#define PARSER_H
```

```
#include "lexer.h"
```

```
bool check_syntax(Token* ll, VariableType curr);
```

```
typedef enum {
```

```
    LINE,
```

```
    BG,
```

```
    TGT,
```

```
    CMD,
```

```
    ARG
```

```
} VariableType;
```

```
#endif #include "lexer.h"
```

```
#include "parser.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
bool check_syntax(Token* ll, VariableType currVariable){
```

```
    TokenType currTerminal = ll->type;
```

```
    while(ll->next!=NULL){
```

```
        if(currVariable==LINE){
```

```
            if(currTerminal==TOKEN_WORD){
```

```
                currVariable = ARG;
```

```
            }else if(currTerminal==NULL){
```

```
            }else{
```

```
                perror("invalid syntax");
```

```
    return false;
}
```

```
}else if(currVariable==ARG){

    if(currTerminal==TOKEN_WORD){
        currVariable = ARG;

    }else if(currTerminal==TOKEN_LT){
        currVariable = TGT;

    }else if(currTerminal==TOKEN_GT){
        currVariable = TGT;

    }else if(currTerminal==TOKEN_GTGT){
        currVariable = TGT;

    }else if(currTerminal==TOKEN_AMP){
        currVariable = BG;

    }else if(currTerminal==TOKEN_SEMI){
        currVariable = CMD;

    }else if(currTerminal==TOKEN_PIPE){
        currVariable = CMD;

    }else{
        perror("invalid syntax");
        return false;
    }

}else if(currVariable==CMD){
    if(currTerminal==TOKEN_WORD){
        currVariable = ARG;

    }else{
        perror("invalid syntax");
```

```

        return false;
    }

} else if (currVariable == TGT) {
    if (currTerminal == TOKEN_WORD) {
        currVariable = ARG;

    } else {
        perror("invalid syntax");
        return false;
    }

} else if (currVariable == BG) {

    if (currTerminal == TOKEN_WORD) {
        currVariable = ARG;

    } else if (currTerminal == NULL) {

    } else {
        perror("invalid syntax");
        return false;
    }

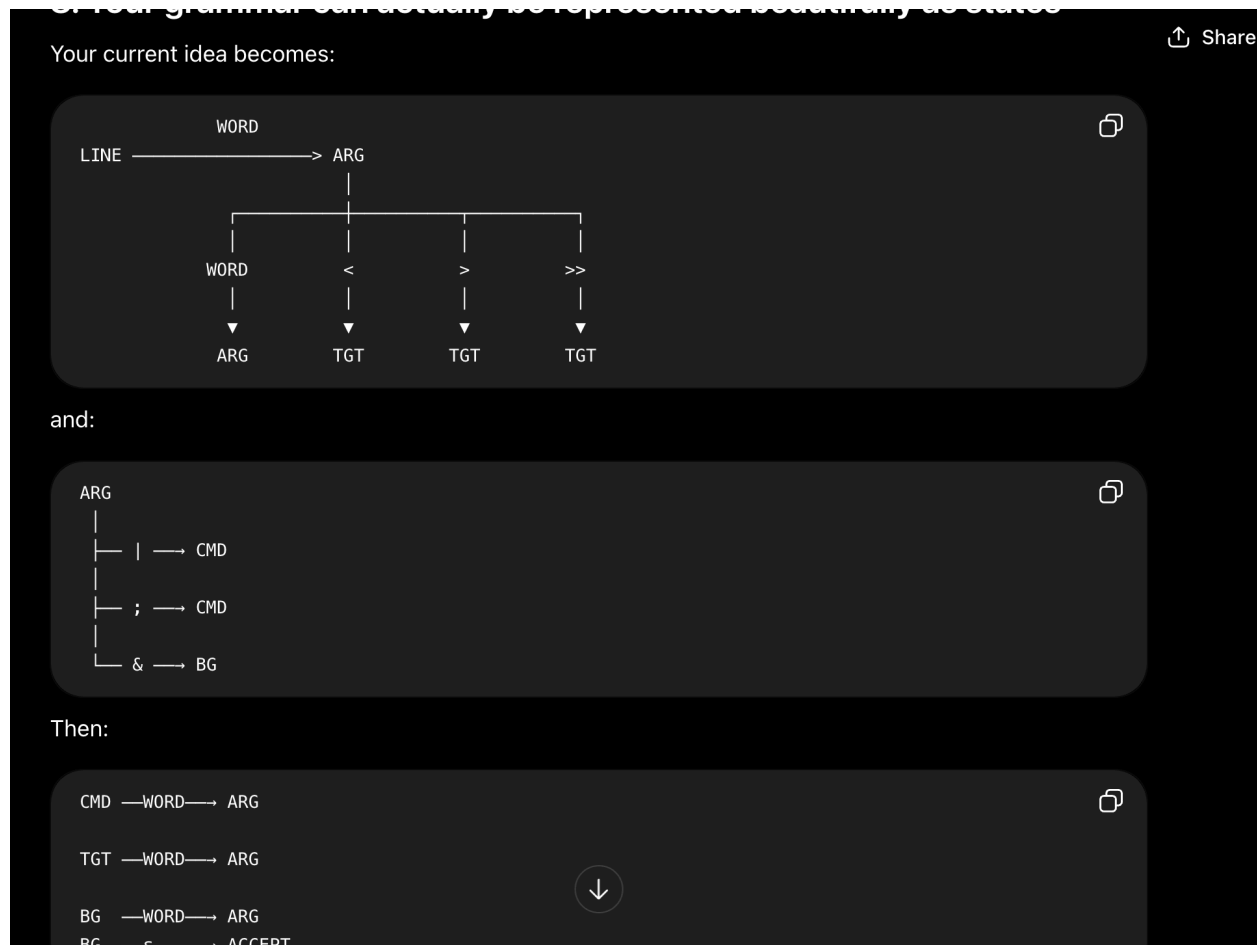
} else {
    perror("invalid syntax");
    return false;
}

ll = ll->next;
currTerminal = ll->type;

}

return true;
}

```

10. One thing I'd change in your thinking

You called it:

```
check_syntax(Token *ll, VariableType curr)
```

That's okay, but conceptually what you're building is a parser state machine.

I'd think of the function as:

current grammar variable

+

current token

↓

next grammar variable

For example:

(ARG, WORD)



(ARG)

(ARG, OP_PIPE)



(CMD)

(CMD, WORD)



(ARG)

(CMD, OP_GT)



INVALID

Prompt 9 :

donot use switch use if else

```

bool check_syntax(Token *tokens, VariableType current)
{
    while (tokens != NULL) {
        TokenType token = tokens->type;

        if (current == LINE) {
            if (token == TOKEN_WORD) {
                current = ARG;
            } else {
                return false;
            }
        } else if (current == ARG) {
            if (token == TOKEN_WORD) {
                current = ARG;
            } else if (token == TOKEN_LT ||
                       token == TOKEN_GT ||
                       token == TOKEN_GTGT) {
                current = TGT;
            } else if (token == TOKEN_PIPE ||
                       token == TOKEN_SEMI) {
                current = CMD;
            } else if (token == TOKEN_AMP) {
                current = BG;
            } else {
                return false;
            }
        } else if (current == CMD) {
            if (token == TOKEN_WORD) {
                current = ARG;
            } else {
                return false;
            }
        }
    }
}

```

```

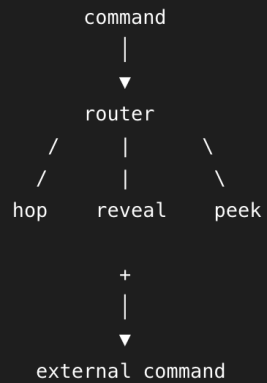
graph TD
    A[User input] --> B[read_input()]
    B --> C[lex_line()]
    C --> D[Token linked list]
    D --> E[check_syntax(tokens, LINE)]
    E --> F{ }
    F -- valid --> G[continue]
    F -- invalid --> H[print error]

```

Prompt 10 :

Asked about B1,B2

no we will talk about how to change the cwd like how should i move into another dir



The most important OS concept

You might be wondering:

"Why can't I just call `chdir()` in a child process?"

You **must not** do that for `hop`.

Suppose:



The child changes its own CWD.

The parent shell remains where it was.

So after the child exits:

Shell CWD = unchanged

Therefore built-ins like `hop` must execute **inside the shell process itself**.

This is a fundamental shell concept.



External commands

This is a fundamental shell concept.

External command:

```
shell
|
└─ fork → child → exec(command)
      ↓
    exits
```

hop:

```
shell
|
└─ chdir() ← directly in shell
```

That's why your router will eventually do something like:

```
router
|
├─ hop → execute inside shell
├─ reveal → execute inside shell
├─ peek → execute inside shell
└─ locate → execute inside shell
```

Prompt 11 :

If you want to keep your current design for now

You *can* do:

prompt.h

```
extern char *home_directory;
```

prompt.c

```
char *home_directory;
```

Then initialize it when the shell starts:

```
home_directory = getcwd(...);
```

And **hop.c** can include:

```
#include "prompt.h"
```

and use:

```
chdir(home_directory);
```

The important distinction is:

```
// prompt.h
extern char *home_directory;
```

does **not allocate** the variable.

It says:

"There is a variable called `home_directory` somewhere else."

Then:

```
// prompt.c
char *home_directory;
```

actually **defines** it.

But I'd recommend one small refactor

Since you're about to implement several shell intrinsics, create:

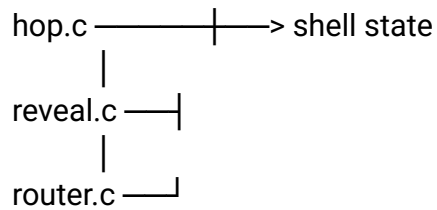
```
shell.h
shell.c
```

and put shell-wide state there:

```
shell.c
    home_directory
    previous_directory
    ...
```

Then:

```
prompt.c ─┐
          │
```



That will become much cleaner once you implement `hop` - and frequency.

For now, though, **your `prompt.h` approach will work**. If you want to avoid changing your architecture unnecessarily at this stage, use `extern` in `prompt.h` and define the variable in `prompt.c`.

Prompt 12 :

Complete the code

```
#include "hop.h"
#include "shell.h"
#include "lexer.h"
```

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```
static char *get_current_directory(void)
{
    char buffer[PATH_MAX];

    if (getcwd(buffer, sizeof(buffer)) == NULL) {
        return NULL;
    }
}
```

```
char *directory = strdup(buffer);
```

```
if (directory == NULL) {
    perror("strdup");
    exit(EXIT_FAILURE);
}
```

```
return directory;
```

```
}
```

```
static bool change_directory(const char *path)
```

```
{
```

```
    char *current = get_current_directory();
```

```
    if (current == NULL) {
```

```
        return false;
```

```
    }
```

```
    if (chdir(path) == -1) {
```

```
        free(current);
```

```
        return false;
```

```
    }
```

```
    free(shell.previous_directory);
```

```
    shell.previous_directory = current;
```

```
    return true;
```

```
}
```

```
void hop_handler(Token *values)
```

```
{
```

```
    while (values != NULL) {
```

```
        char *value = values->value;
```

```
        if (strcmp(value, "~") == 0) {
```

```
            change_directory(shell.home_directory);
```

```
        } else if (strcmp(value, ".") == 0) {
```

```
            /* Do nothing. */
```

```
        } else if (strcmp(value, "..") == 0) {
```

```
            change_directory("..");
```

```
        } else if (strcmp(value, "-") == 0) {
```

```
            if (shell.previous_directory != NULL) {
```

```
                change_directory(shell.previous_directory);
```

```
            }
```



```

    } else {
        if (!change_directory(value)) {
            printf("hop: no such directory\n");
        }
    }

    values = values->next;
}
}

```

Prompt 13 :
am initailizing home directory in prompt.c v

If `init_prompt()` already initializes `shell.home_directory`, make sure it doesn't overwrite it every time you print the prompt.

You want:

```

<> C
init_prompt();    // once
while (...) {
    print_prompt(); // many times
}

```

not:

```

<> C
while (...) {
    init_prompt(); // ❌ would reset state
    print_prompt();
}

```

Because later `shell.previous_directory` and your frequency state need to persist throughout the shell session.

So your current architecture is perfectly reasonable:

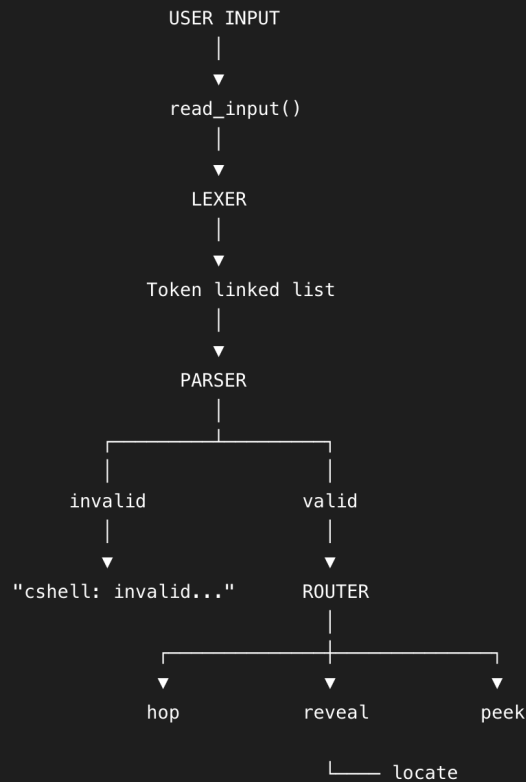
```

prompt.c
├──
└── initializes home_directory

```

Prompt 14 :
Is my architecture till now correct

Your overall architecture is now



One thing to remember: **don't free the tokens before calling `router()`**. The router needs the list to identify the command and pass its arguments to the appropriate handler.

Prompt 15 :

Debug wait then we will move back to hop.c my current hop.c is and hop.h is #include "lexer.h"

```
#ifndef HOP_H
```

```
#define HOP_H
```

```
void hop_handler(Token* values);
```

```
#endif #include "hop.h"
```

```
#include "lexer.h"
```

```
#include "shell.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void hop_handler(Token* values){
```

```
    while(values!=NULL){
```

```
        char *value = values->value;
```

```
        if(strcmp(value, "~") == 0){
```

```
            shell.previous_directory = getcwd();
```

```
            chdir(shell.home_directory);
```

```
        }else if(strcmp(values->value, ".") == 0){
```

```
        }else if(strcmp(values->value, "..") == 0){
```

```
        }else if(strcmp(values->value, "-") == 0){
```

```
        }else if(strcmp(values->value, ".") == 0){
```

```
        }else{
```

```
            chdir(value);
```

```
        }
```

```
        values = values->next;
```

```
    }
```

```
return ;
```

```
}
```

Prompt 17 :

`free(shell.previous_directory); shell.previous_directory = current;` why freeing and allocating again

Why can't we just store the current path without `getcwd()` ?

Because `previous_directory` needs to contain a persistent copy of the old path.

After:

```
</> C
chdir(path);
```

the process's CWD changes. We can't somehow ask the OS later:

```
"What was my previous CWD?"
```

The OS doesn't maintain that shell-level concept for us.

So we save the old path ourselves **before** changing it.

That's the fundamental idea:

```
getcwd()
  ↓
save old CWD
  ↓
chdir(new)
  ↓
previous_directory = saved old CWD
```

And because `getcwd(NULL, 0)` gives us dynamically allocated memory, we have to manage that memory with `free()`.

Prompt 18 :

1. If the current working directory has the home directory as an ancestor (e.g. `/path/to/home/osn`), replace the home directory prefix with `~`. So the example becomes `~/osn`. considering this is this output valid
`<abhilashreddy@Abhilash-ka-MacBook-Air.local:~> hop ..`
`<abhilashreddy@Abhilash-ka-MacBook-Air.local:/Users/abhilashreddy/Desktop/IIITH`
`/sem3/OSN/mini-project1>`

No — if `hop ..` actually succeeded, that output is not valid according to A1.

Your shell's home directory is:

/Users/abhilashreddy/Desktop/IIITH/sem3/OSN/mini-project1

After:

hop ..

the CWD should become:

/Users/abhilashreddy/Desktop/IIITH/sem3/OSN

Now apply the A1 rule:

Replace the home-directory prefix with ~ **only if the current CWD has the home directory as an ancestor.**

Is:

/Users/abhilashreddy/Desktop/IIITH/sem3/OSN

inside:

/Users/abhilashreddy/Desktop/IIITH/sem3/OSN/mini-project1

?

No.

The relationship is the opposite:

/Users/abhilashreddy/Desktop/IIITH/sem3/OSN

└─ mini-project1 ← home

Your home is a **child** of the current directory, not an ancestor of it.

Therefore the prompt should display the **absolute path**:

<abhilashreddy@Abhilash-ka-MacBook-Air.local:/Users/abhilashreddy/Desktop/IIITH/sem3/OSN>

So this:

<abhilashreddy@Abhilash-ka-MacBook-Air.local:~>

hop ..

<abhilashreddy@Abhilash-ka-MacBook-Air.local:/Users/abhilashreddy/Desktop/IIITH/sem3/OSN>

is **correct**.

Your prompt implementation's condition:

```
strncmp(cwd, shell_home, strlen(shell_home)) == 0
```

plus:

```
cwd[strlen(shell_home)] == '/'
```

correctly prevents `/.../OSN` from being treated as `~/...`

So your prompt behavior here is right.

14. The architecture I'd use

So B2 becomes:

```
reveal_handler()
├── parse flags
│   ├── -a
│   └── -t
├── identify directory argument
├── resolve directory
│   ├── ~
│   ├── .
│   ├── ..
│   ├── -
│   └── normal path
└── list_directory()
    ├── opendir()
    ├── readdir()
    ├── filter hidden
    ├── sort
    ├── print
    └── recurse if -t
```

Prompt 19 :

now tell me which files to create and how to code

```
c-shell/
├── include/
│   ├── input.h
│   ├── lexer.h
│   ├── parser.h
│   ├── prompt.h
│   ├── router.h
│   ├── shell.h
│   ├── hop.h
│   ├── peek.h
│   ├── locate.h
│   ├── reveal.h      ← already/now
│   └── path_utils.h  ← NEW
└── src/
    ├── main.c
    ├── input.c
    ├── lexer.c
    ├── parser.c
    ├── prompt.c
    ├── router.c
    ├── shell.c
    ├── hop.c
    ├── reveal.c      ← NEW/implementation
    └── path_utils.c  ← NEW
```

2. path_utils.c

We'll start with the five cases:

```
~    → home
.    → current directory
..   → parent
-    → previous directory
other → direct path
#include "path_utils.h"
#include "shell.h"
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```

char *resolve_path(const char *path)
{
    if (strcmp(path, "~") == 0) {
        return strdup(shell.home_directory);
    }

    if (strcmp(path, ".") == 0) {
        return getcwd(NULL, 0);
    }

    if (strcmp(path, "..") == 0) {
        char *current = getcwd(NULL, 0);

        if (current == NULL) {
            return NULL;
        }

        char *slash = strrchr(current, '/');

        if (slash != NULL) {
            if (slash == current) {
                slash[1] = '\0';
            } else {
                *slash = '\0';
            }
        }

        return current;
    }

    if (strcmp(path, "-") == 0) {
        if (shell.previous_directory == NULL) {
            return NULL;
        }

        return strdup(shell.previous_directory);
    }

    return strdup(path);
}

```


One thing I want you to notice

We haven't written the whole B2 in one giant function.

We're building it in layers:

```
Layer 1
argument parsing
  ↓
Layer 2
path resolution
  ↓
Layer 3
directory reading
  ↓
Layer 4
sorting
  ↓
Layer 5
hidden-file filtering
  ↓
Layer 6
recursive traversal
```

Prompt 21 :

kk wait before going into coding of path_utils hop is already working perfectly with the use of shell why does it need path_utils

Exactly. It **doesn't need path_utils right now**. If your `hop.c` is working correctly using `shell.home_directory` and `shell.previous_directory`, we shouldn't refactor it just for the sake of abstraction.

I jumped ahead by suggesting `path_utils`. Your instinct is better here: **don't introduce a common abstraction until we actually need it**.

For B2, we can implement `reveal` independently.

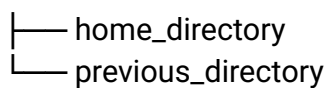
Current architecture

Keep:

`shell.c`



`ShellState`



And:

hop.c



uses ShellState



chdir()

Then:

reveal.c



uses ShellState

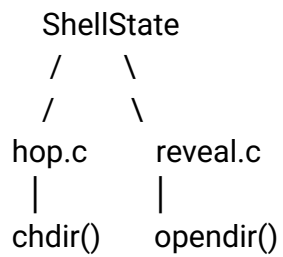


resolves path



opendir()

So:



No `path_utils.c` needed.

Why reveal can just resolve things itself

For example:

reveal ~

can directly use:

shell.home_directory

while:

reveal -

can directly use:

shell.previous_directory

and:

reveal .

can use:

getcwd(NULL, 0)

and:

reveal ..

can derive the parent.

For a normal path:

reveal src

we can simply give "src" to:

opendir("src");

The OS handles the relative path.

So **let's not create path_utils.h/c.**

Your files remain:

```
include/  
├── shell.h  
├── hop.h  
├── reveal.h  
└── ...
```

```
src/  
├── shell.c  
├── hop.c  
├── reveal.c  
└── ...
```

And we'll build **reveal.c** from scratch around your existing shell state.

The next logical step is to design how **reveal** should parse **-a, -t**, the optional directory, and reject invalid combinations before touching **opendir()**.

Prompt 22 :

now iam telling utils will be used for reveal kk but hop is already using shell iam asking why we should replace it

Ah, yes — I understand what you're asking now.

We don't need to replace **hop's existing implementation with **path_utils**.**

You can absolutely keep:

hop.c
↓
shell.home_directory
shell.previous_directory
↓
chdir()

and introduce **path_utils** only for **reveal** if it makes **reveal** cleaner.

There is no requirement that both commands use the same helper.

Why?

hop and **reveal** have different jobs.

hop:

resolve destination
↓
chdir(destination)
↓
update previous_directory

Your current **hop.c** already does this correctly.

reveal:

resolve directory
↓

DO NOT `chdir()`

↓

`opendir(directory)`

↓

list contents

So a utility like:

`resolve_path()`

is potentially useful for `reveal`, because `reveal` needs to turn:

~

.

..

-

name

into something it can pass to `opendir()` without changing the shell's CWD.

Therefore

Keep your current:

`hop.c`

exactly as it is.

Create:

`path_utils.c`

`path_utils.h`

only if we find that `reveal.c` benefits from it.

And there's actually another good reason to do this: if later we notice that `locate` or some other feature needs the same path resolution, then the utility becomes genuinely reusable.

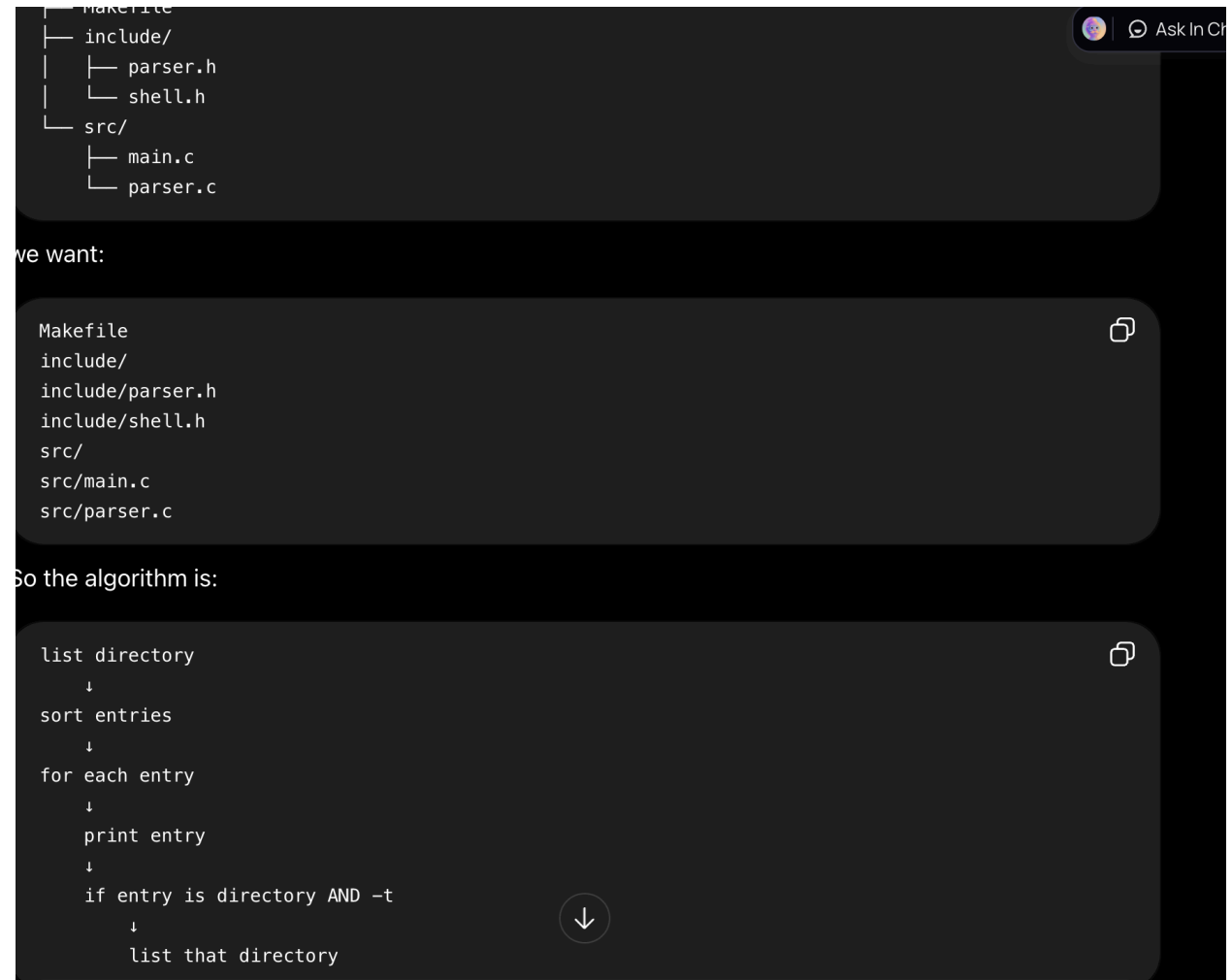
So the principle is:

Don't refactor working code just to use a new abstraction. Introduce the abstraction where it solves a real problem.

Your **hop** is already working → **leave it alone.**

Prompt 23 :

implemet recursive



The screenshot shows a code editor with a file tree on the left and a code block on the right. The file tree shows a project structure with a Makefile, include/ directory (containing parser.h and shell.h), and src/ directory (containing main.c and parser.c). The code block contains a recursive algorithm diagram. The diagram shows a flow from 'list directory' to 'sort entries', then a loop 'for each entry' which leads to 'print entry'. A conditional 'if entry is directory AND -t' leads to 'list that directory', which loops back to 'list directory'. There are copy icons next to the code blocks.

```
Makefile
include/
  parser.h
  shell.h
src/
  main.c
  parser.c
```

we want:

```
Makefile
include/
  include/parser.h
  include/shell.h
src/
  src/main.c
  src/parser.c
```

So the algorithm is:

```
list directory
  ↓
sort entries
  ↓
for each entry
  ↓
  print entry
  ↓
  if entry is directory AND -t
    ↓
    list that directory
```

```
#include "reveal.h"
#include "path_utils.h"
```

```
#include <dirent.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
```

```
static bool is_flag(const char *value)
{
    return value[0] == '-' && value[1] != '\0';
}
```

```
static bool parse_flag(
    const char *value,
    bool *show_hidden,
    bool *recursive
)
{
    for (size_t i = 1; value[i] != '\0'; i++) {
        if (value[i] == 'a') {
            *show_hidden = true;
        } else if (value[i] == 't') {
            *recursive = true;
        } else {
            return false;
        }
    }

    return true;
}
```

```
static int compare_names(const void *a, const void *b)
{
    const char *first = *(const char **)a;
    const char *second = *(const char **)b;

    return strcmp(first, second);
}
```

```
static void free_names(char **names, size_t count)
{
    for (size_t i = 0; i < count; i++) {
        free(names[i]);
    }

    free(names);
}
```

```

static void list_directory(
    const char *path,
    bool show_hidden,
    bool recursive
)
{
    DIR *directory = opendir(path);

    if (directory == NULL) {
        return;
    }

    char **names = NULL;
    size_t count = 0;
    size_t capacity = 0;

    struct dirent *entry;

    while ((entry = readdir(directory)) != NULL) {
        if (!show_hidden && entry->d_name[0] == '.') {
            continue;
        }

        if (count == capacity) {
            if (capacity == 0) {
                capacity = 16;
            } else {
                capacity *= 2;
            }

            char **new_names = realloc(
                names,
                capacity * sizeof(*names)
            );

            if (new_names == NULL) {
                perror("realloc");
                closedir(directory);
                free_names(names, count);
            }
        }
    }
}

```



```

        exit(EXIT_FAILURE);
    }

    names = new_names;
}

names[count] = strdup(entry->d_name);

if (names[count] == NULL) {
    perror("strdup");
    closedir(directory);
    free_names(names, count);
    exit(EXIT_FAILURE);
}

count++;
}

closedir(directory);

qsort(
    names,
    count,
    sizeof(*names),
    compare_names
);

for (size_t i = 0; i < count; i++) {
    printf("%s%s\n", names[i],
        recursive && names[i][strlen(names[i]) - 1] != '/'
        ? ""
        : "");

    if (!recursive) {
        continue;
    }

    if (strcmp(names[i], ".") == 0 ||
        strcmp(names[i], "..") == 0) {
        continue;
    }
}

```

```

    }

    char *child_path = malloc(
        strlen(path) + strlen(names[i]) + 2
    );

    if (child_path == NULL) {
        perror("malloc");
        free_names(names, count);
        exit(EXIT_FAILURE);
    }

    sprintf(
        child_path,
        "%s/%s",
        path,
        names[i]
    );

    struct stat info;

    if (stat(child_path, &info) == 0 &&
        S_ISDIR(info.st_mode)) {

        printf("%s/\n", names[i]);

        list_directory(
            child_path,
            show_hidden,
            recursive
        );
    }

    free(child_path);
}

free_names(names, count);
}

void reveal_handler(Token *values)

```

```

{
    bool show_hidden = false;
    bool recursive = false;

    char *directory = NULL;

    while (values != NULL) {
        char *value = values->value;

        if (is_flag(value) && strcmp(value, "-") != 0) {
            if (!parse_flag(value, &show_hidden, &recursive)) {
                printf("reveal: invalid syntax\n");
                free(directory);
                return;
            }
        } else {
            if (directory != NULL) {
                printf("reveal: invalid syntax\n");
                free(directory);
                return;
            }

            directory = strdup(value);

            if (directory == NULL) {
                perror("strdup");
                exit(EXIT_FAILURE);
            }
        }

        values = values->next;
    }

    if (directory == NULL) {
        directory = strdup(".");
    }

    char *path = resolve_path(directory);

    free(directory);

```

```

if (path == NULL) {
    printf("reveal: no such directory\n");
    return;
}

struct stat info;

if (stat(path, &info) == -1 ||
    !S_ISDIR(info.st_mode)) {
    printf("reveal: no such directory\n");
    free(path);
    return;
}

list_directory(
    path,
    show_hidden,
    recursive
);

free(path);
}

```

Prompt 24 :
 now reexplain me everything how reveal is done

That new call independently:

1. opens `include`
2. reads entries
3. sorts them
4. prints them
5. recursively enters any directories

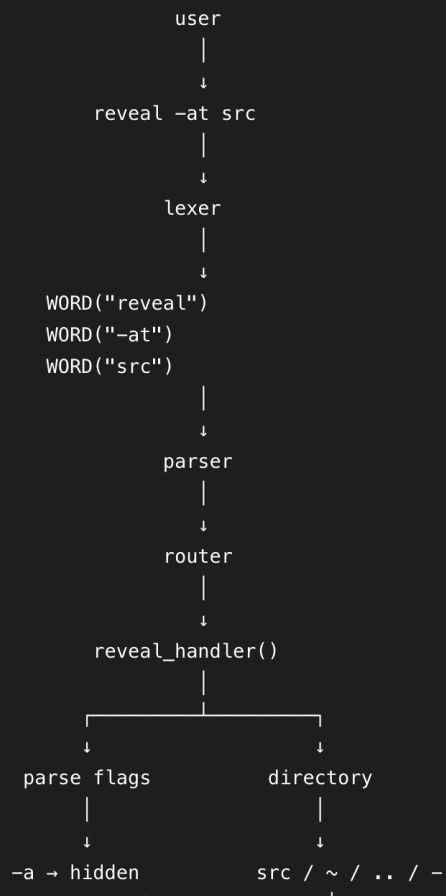
So:

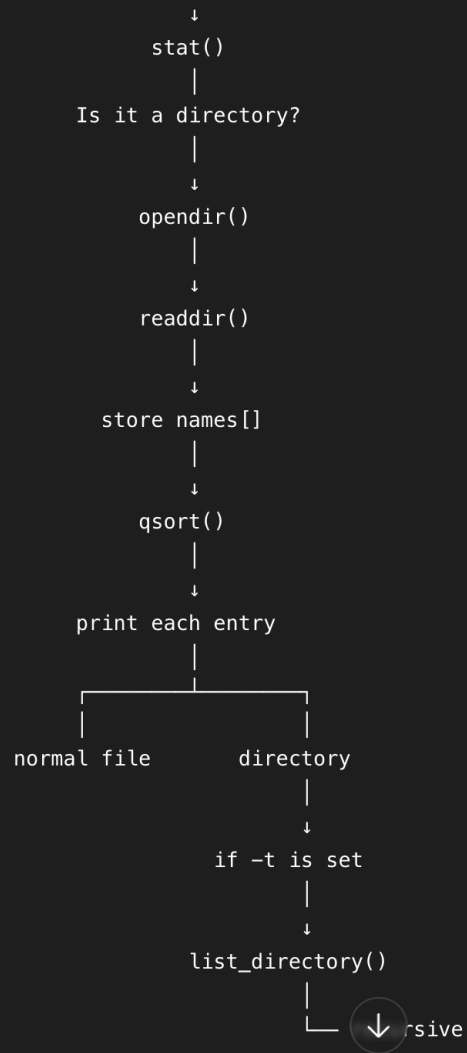
```
list_directory(c-shell)
  ↓
  include/
  ↓
  list_directory(include)
  ↓
  lexer.h
  parser.h
  ↓
  return
  ↓
  src/
  ↓
  list_directory(src)
  ↓
  main.c
  parser.c
```



21. Overall architecture

Your entire B2 flow is:





For part C :

<https://chatgpt.com/share/6a8d82d0-a5e8-83e9-a822-0b9ade49c151>